SHIV NADAR UNIVERSITY
CHENNAI

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

SCHOOL OF ENGINEERING

PROJECT REPORT

PING-PONG GAME POWERED USING ARDUINO UNO

A PROJECT REPORT

Submitted by

J Sidtharth (24011102037)
M Rathish Roshaan (24011102051)
M Udhaya (24011102053)

Introduction to Internet of Things and Laboratory

SHIV NADAR UNIVERSITY

MAY 2025

SHIV NADAR UNIVERSITY

Ping-Pong Game:

Table of Contents:

SHIV NADAR UNIVERSITY

Abstract:

The RUS PONG GAME is a microcontroller-based interactive arcade game developed using the Arduino UNO and a 0.96" OLED display (Figure 3 and 5). The game replicates the functionality of the classic Pong game with two-player control, real-time paddle motion, bouncing ball mechanics, and dynamic scoring. Built with the objective of utilizing minimal hardware and open-source libraries, the game demonstrates the integration of display module, joysticks (Figure 4) in an embedded system. The OLED screen renders game graphics smoothly, and the entire gameplay is managed within the constraints of the ATmega328P microcontroller. This project serves both educational and gaming purposes and is suitable for embedded systems learning, beginner game development, and microcontroller interfacing.

Introduction:

Overview:

Pong, one of the earliest arcade video games, introduced players to digital interactive entertainment in 1972. Originally developed by Atari, the game involved two paddles and a ball bouncing across the screen, mimicking table tennis. In this project, the same principle has been applied using modern embedded hardware—the Arduino UNO microcontroller—paired with a 128x64 OLED display (Figure 5). The aim is to create a compact, interactive version of Pong that can run independently on a microcontroller without external computing resources. By leveraging open-source libraries such as Adafruit's SSD1306 and GFX libraries, real-time graphics rendering is accomplished efficiently on a resource-constrained platform.

This game not only serves as an engaging interactive system but also demonstrates fundamental concepts in embedded programming, such as real-time display management, basic game physics, and peripheral interfacing. The development of RUS PONG GAME (Figure 5) provides valuable insights into designing time-sensitive applications using microcontrollers and highlights how retro games can be recreated using simple and affordable electronics.

Objectives:

1. To develop a two-player interactive Pong game using an Arduino UNO and an OLED display, showcasing the potential of embedded systems in retro-style game development.
2. To improve real-time input-response interaction, using joysticks as game controllers for paddle movement.
3. To create a flicker-free visual display using the `Adafruit_SSD1306` and `Adafruit_GFX` libraries [3], enhancing gameplay smoothness and user experience.
4. To build a complete standalone embedded system that demonstrates game logic, frame control, collision detection, and scoring – all within the limited resources of an Arduino UNO.
5. To foster educational learning through hands-on experience in C/C++ programming, microcontroller interfacing, and display handling.

Motivation:

The interest in retro gaming and the educational value of recreating classic games using modern hardware platforms inspired the development of the RUS PONG GAME. By implementing Pong on the Arduino UNO, the project offers a hands-on approach to understanding microcontroller programming, display interfacing, and real-time system design. This endeavor not only pays homage to the origins of video gaming but also serves as a practical exercise in embedded systems development [4] fostering a deeper appreciation for the intricacies involved in game design and hardware integration.

Functionalities:

1. Two-Player Mode:
   The game supports two players simultaneously, where each player controls a paddle on either side of the OLED screen. Movement is handled via joysticks.

2. Real-Time Ball Movement and Physics:
   The game simulates basic ball dynamics—movement, speed, and angle of reflection—allowing it to bounce off walls and paddles realistically, with varying trajectories based on collision points.

3. Score Tracking System: [3]
   A simple scoring mechanism is implemented and displayed on the OLED. Each time a player misses the ball, the opponent scores a point.

4. Game Over Detection and Reset Option:
   Once a player reaches the maximum score (15), the game stops, displays a win/lose message, and waits for a reset trigger from the player. It also displays MATCHPOINT when either of the players achieve 14 and are about to win in the next addition.

5. Minimalist User Interface: [4]
   The game utilizes an OLED display to present a clean, retro interface that mimics the classic arcade Pong style. Clear rendering and fast refresh rate provide a satisfying gameplay experience.

Literary Survey: (2)

1. Basic Programming Structure Using a Microcontroller System: Ping-Pong Game

Restrepo-Alvarez and Navas [1] present the design and implementation of a ping-pong game on a 7x5 LED matrix using a microcontroller system. The project serves as a pedagogical tool for teaching basic programming concepts, encompassing software development processes and algorithmic structures.

2. Design and Implementation of Multi-Player Pong Game Using Altera DE2 Board

Saha et al. [2] detail the development of an FPGA-based Pong game implemented on an Altera DE2 board using Verilog HDL. The system includes VGA controllers, object creation and animation, and text subsystems, culminating in a functional circuit. Notably, the game features both single-player and multi-player modes, with the latter incorporating real-time and automatic players to enhance competitiveness. The design efficiently utilizes logic elements, with minimal usage percentages across different modes, thereby reducing processing time and making it cost-effective for widespread application.

3. Enriching Computer Science Programming Classes with Arduino Game Development

Duch and Jaworski [3] explore the integration of Arduino-based game development into computer science programming education. They introduce an electronic education board equipped with various input/output devices, such as joysticks, displays, and keypads, to facilitate interactive learning. This approach enables students, regardless of prior electronics experience, to construct simple games and applications, thereby extending their programming capabilities beyond traditional console-based exercises.

4. Machine Learning with the Pong Game: A Case Study

Logofătu [4] investigates the application of machine learning techniques to develop a self-playing agent for the classic Pong game. The study summarizes notable approaches in artificial intelligence and machine learning to create agents capable of competing against human players. Additionally, it proposes a template for evolving this concept into a comprehensive application, with an implementation available in Java.

5. Design of a Smart Ping Pong Robot

Ansari et al. [5] present the design of a smart ping pong robot intended for table tennis applications, allowing players to practice without a human opponent. The robot autonomously launches balls, enabling users to engage in practice sessions. The design emphasizes high performance at a low cost, utilizing mechanical components to circumvent issues commonly associated with electronic platforms.

Proposed Methodology:

Block diagram:

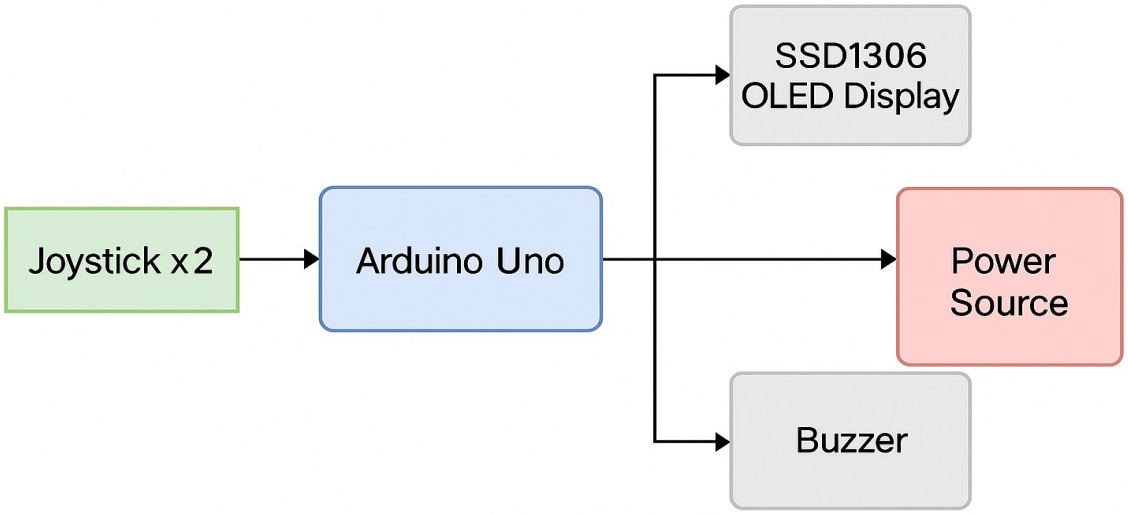

Figure 1: Block diagram of the Pong Game (Buzzer – Optional)

Circuit Diagram:

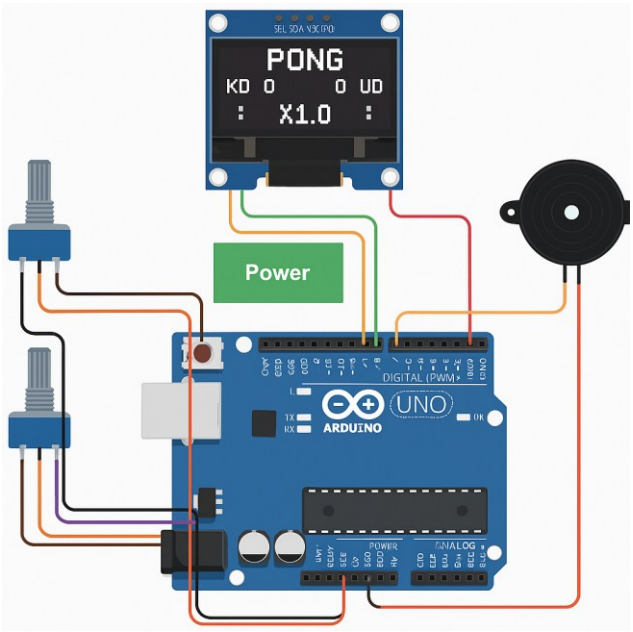

Figure 2: Circuit Diagram of the RUS Pong Game

Pin Diagrams:

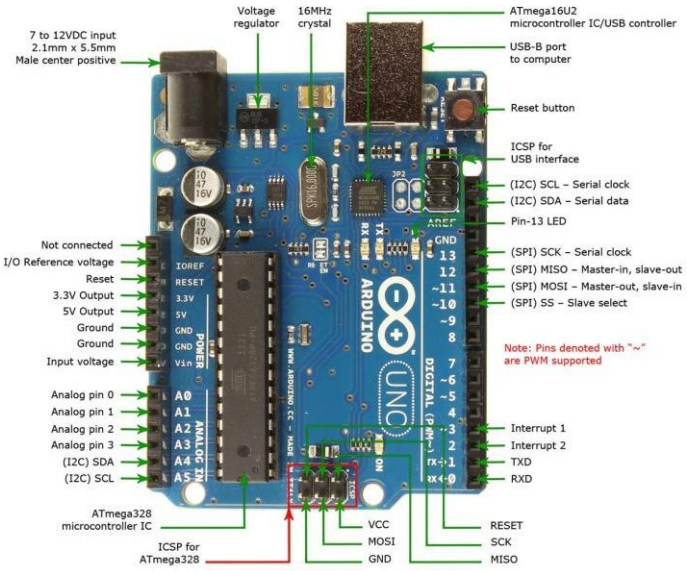

Figure 3: Pin diagram of the Arduino UNO.

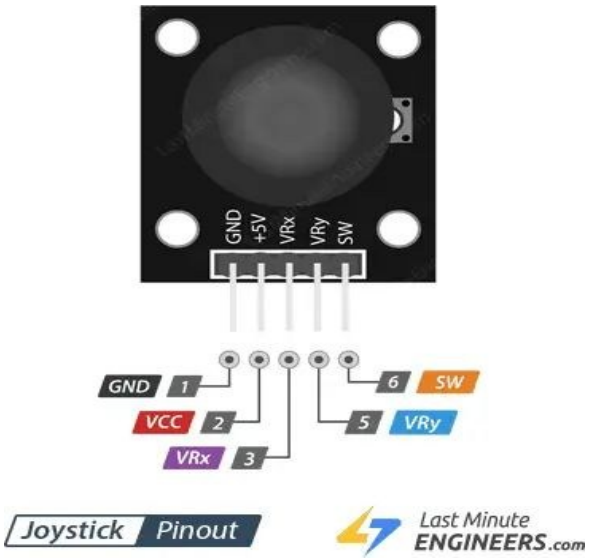

Figure 4: Pin Diagram of the Joysticks used.

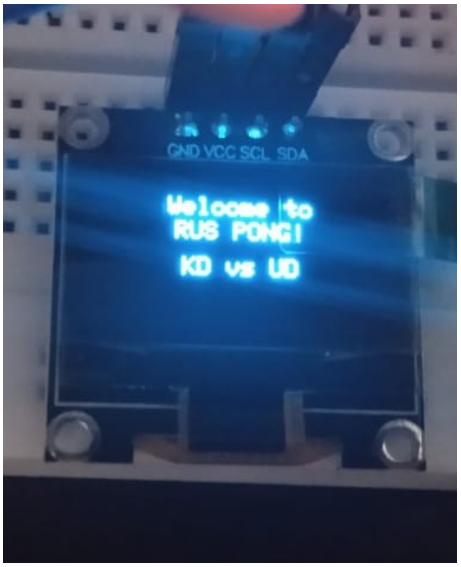

Figure 5: Pin Diagram of the SSD-1306 Display Module (0.96 inches, 128 * 64)

Technical Specifications:

Hardware:

S.No	Component	Quantity
1	Arduino UNO (Figure 1)	1
2	Joysticks (Figure 4)	2
3	Breadboard + Wires	As required
4	USB Cable + Power Source	1
5	SSD 1306 OLED Display 128x64 (Figure 5)	1

Software:

1. Arduino IDE (v1.8+ or v2.x):
   The development environment used to write, compile, and upload code to the Arduino board.

2. Adafruit_SSD1306 Library:
   Handles the OLED display rendering including drawing shapes, text, and graphics.
   - GitHub Link (https://github.com/adafruit/Adafruit_SSD1306)

3. Adafruit_GFX Library:
   A core graphics library required by Adafruit_SSD1306 to handle graphical primitives like lines and circles.
   - GitHub Link (https://github.com/adafruit/Adafruit-GFX-Library)

4. Optional – Tone Library (for buzzer):
   If a buzzer is used, this library can generate tones on a digital pin.

5. Programming Language:
   The logic is written in Arduino C/C++ with structured code flow including setup and loop functions, delay handling, and real-time collision detection.

Methodology of the Project:

The core methodology of the Pong Game project involves the integration of user inputs, game logic, and display handling, all managed through the Arduino UNO microcontroller. The process begins with hardware setup, where the OLED display is connected to the Arduino via the I2C interface (SDA and SCL), and two joysticks are interfaced to allow each player to control their respective paddles. The optional buzzer can be added to provide audio feedback upon scoring events or collisions for enhanced user experience.

Once the hardware is configured, the software logic is implemented using the Arduino IDE. The program initializes the display using the **Adafruit_SSD1306** and **Adafruit_GFX** libraries. The setup () function is used to configure the I/O pins and initialize the display. The loop () function forms the main game engine — it continuously reads button states to detect paddle movement, updates the ball's position, checks for boundary collisions, and redraws the screen for every frame. Collision detection algorithms are used to determine if the ball has hit a paddle or wall, reversing its direction accordingly. If the ball passes a paddle without collision, the opposing player scores a point, and the game resets the ball's position.
Game logic is handled in real-time, using non-blocking code and timed delays for smooth ball movement and paddle control. The score is constantly updated and displayed on the screen. Optionally, the buzzer is activated during events like hitting the paddle or scoring, using the tone () function.

This project emphasizes real-time embedded control, graphical interface handling, and fundamental game logic implementation, forming a comprehensive application of Arduino programming and interfacing skills. Refer Figure 6 for reference.

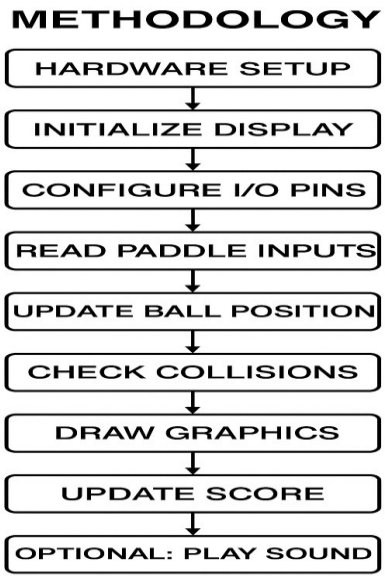

Figure 6: A Flowchart of the Methodology of the Project Flow

Program Code:

```
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
#define OLED_RESET 4
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET);

const int leftJoystick = A0;
const int rightJoystick = A1;
const int PADDLE_HEIGHT = 12;
const int BALL_SIZE = 4;
const int WIN_SCORE = 15;
const float MAX_SPEED = 3.0;

int leftScore = 0;
int rightScore = 0;
int ballX = 64, ballY = 32;
int prevBallX = 64, prevBallY = 32;
float ballSpeedX = 1.0;
float ballSpeedY = 1.0;
int leftPaddle = 26;
int rightPaddle = 26;
unsigned long gameStartTime;
float speedMultiplier = 1.0;
bool isPaused = false;

void setup() {
  display.begin(SSD1306_SWITCHCAPVCC, 0x3C);
  display.clearDisplay();
  display.setTextColor(WHITE);
  display.setTextWrap(false);
  showWelcome();
  countdown();
  gameStartTime = millis();
}

void loop() {
  if (analogRead(leftJoystick) > 1000 && analogRead(rightJoystick) > 1000) {
    isPaused = !isPaused;
    display.clearDisplay();
    centerText("GAME PAUSED", 30);
    display.display();
```

```
    delay(500);
  }
  if (isPaused) return;

  if (leftScore >= WIN_SCORE || rightScore >= WIN_SCORE) {
    gameOver();
    return;
  }

  speedMultiplier = 1.0 + (millis() - gameStartTime) / 30000.0;
  if (speedMultiplier > MAX_SPEED) speedMultiplier = MAX_SPEED;

  movePaddles();
  updateBall();

  drawCourt();
  drawScore();
  drawSpeed();
  drawTimer();
  drawPaddles();
  drawBall();

  if (leftScore == WIN_SCORE - 1 || rightScore == WIN_SCORE - 1) {
    centerText("MATCH POINT!", 52);
  }

  display.display();
  delay(30);
}

void movePaddles() {
  int leftInput = analogRead(leftJoystick);
  if (leftInput < 400) leftPaddle -= 3;
  if (leftInput > 600) leftPaddle += 3;

  int rightInput = analogRead(rightJoystick);
  if (rightInput < 400) rightPaddle -= 3;
  if (rightInput > 600) rightPaddle += 3;

  leftPaddle = constrain(leftPaddle, 0, SCREEN_HEIGHT - PADDLE_HEIGHT);
  rightPaddle = constrain(rightPaddle, 0, SCREEN_HEIGHT - PADDLE_HEIGHT);
}

void updateBall() {
  prevBallX = ballX;
  prevBallY = ballY;
```

```
  ballX += ballSpeedX * speedMultiplier;
  ballY += ballSpeedY * speedMultiplier;

  if (ballY <= 0 || ballY >= SCREEN_HEIGHT - BALL_SIZE) ballSpeedY = -ballSpeedY;

  if (ballX <= 5 && ballY + BALL_SIZE >= leftPaddle && ballY <= leftPaddle + PADDLE_HEIGHT) {
    ballSpeedX = abs(ballSpeedX) * 1.05;
    float hitPos = (ballY + BALL_SIZE / 2) - (leftPaddle + PADDLE_HEIGHT / 2);
    ballSpeedY = hitPos / 4.0;
    ballX = 6;
  }

  if (ballX + BALL_SIZE >= SCREEN_WIDTH - 5 && ballY + BALL_SIZE >= rightPaddle && ballY <= rightPaddle +
PADDLE_HEIGHT) {
    ballSpeedX = -abs(ballSpeedX) * 1.05;
    float hitPos = (ballY + BALL_SIZE / 2) - (rightPaddle + PADDLE_HEIGHT / 2);
    ballSpeedY = hitPos / 4.0;
    ballX = SCREEN_WIDTH - 5 - BALL_SIZE;
  }

  if (ballX < 0) {
    rightScore++;
    resetBall();
    delay(500);
  }
  if (ballX > SCREEN_WIDTH) {
    leftScore++;
    resetBall();
    delay(500);
  }
}

void resetBall() {
  ballX = SCREEN_WIDTH / 2;
  ballY = random(20, SCREEN_HEIGHT - 20);
  ballSpeedX = random(0, 2) ? 1.0 : -1.0;
  ballSpeedY = random(-2, 2);
}

void drawCourt() {
  display.clearDisplay();
  display.drawRect(0, 0, SCREEN_WIDTH, SCREEN_HEIGHT, WHITE);
  for (int y = 0; y < SCREEN_HEIGHT; y += 6) {
    display.drawFastVLine(SCREEN_WIDTH / 2, y, 3, WHITE);
  }
}
```

```
void drawScore() {
  display.setTextSize(1);
  display.setCursor(2, 2);
  display.print("KD");
  display.setCursor(2, 10);
  display.print(leftScore);

  display.setCursor(SCREEN_WIDTH - 18, 2);
  display.print("UD");
  display.setCursor(SCREEN_WIDTH - 18, 10);
  display.print(rightScore);
}

void drawSpeed() {
  display.setTextSize(1);
  display.setCursor((SCREEN_WIDTH / 2) - 15, 2);
  display.print("x");
  display.print(speedMultiplier, 1);
}

void drawTimer() {
  display.setTextSize(1);
  unsigned long elapsed = (millis() - gameStartTime) / 1000;
  display.setCursor((SCREEN_WIDTH / 2) - 12, SCREEN_HEIGHT - 8);
  display.print("T:");
  display.print(elapsed);
  display.print("s");
}

void drawPaddles() {
  display.fillRect(2, leftPaddle, 3, PADDLE_HEIGHT, WHITE);
  display.fillRect(SCREEN_WIDTH - 5, rightPaddle, 3, PADDLE_HEIGHT, WHITE);
}

void drawBall() {
  display.fillRect(prevBallX, prevBallY, BALL_SIZE, BALL_SIZE, BLACK);
  display.fillRect(ballX, ballY, BALL_SIZE, BALL_SIZE, WHITE);
}

void showWelcome() {
  display.clearDisplay();
  display.setTextSize(1);
  centerText("Welcome to", 10);
  centerText("RUS PONG!", 20);
  centerText("KD vs UD", 35);
```

```
  display.display();
  delay(2000);
}

void countdown() {
  for (int i = 3; i > 0; i--) {
    display.clearDisplay();
    display.setTextSize(2);
    centerTextLarge(String(i), 20);
    display.display();
    delay(800);
  }
}

void gameOver() {
  display.clearDisplay();
  display.setTextSize(2);
  if (leftScore >= WIN_SCORE)
    centerTextLarge("KD WINS!", 20);
  else
    centerTextLarge("UD WINS!", 20);
  display.display();
  delay(4000);

  leftScore = 0;
  rightScore = 0;
  resetBall();
  gameStartTime = millis();
  countdown();
}

void centerText(String text, int y) {
  int x = (SCREEN_WIDTH - text.length() * 6) / 2;
  display.setCursor(x, y);
  display.print(text);
}

void centerTextLarge(String text, int y) {
  int x = (SCREEN_WIDTH - text.length() * 12) / 2;
  display.setCursor(x, y);
  display.print(text);
}
```

Results:

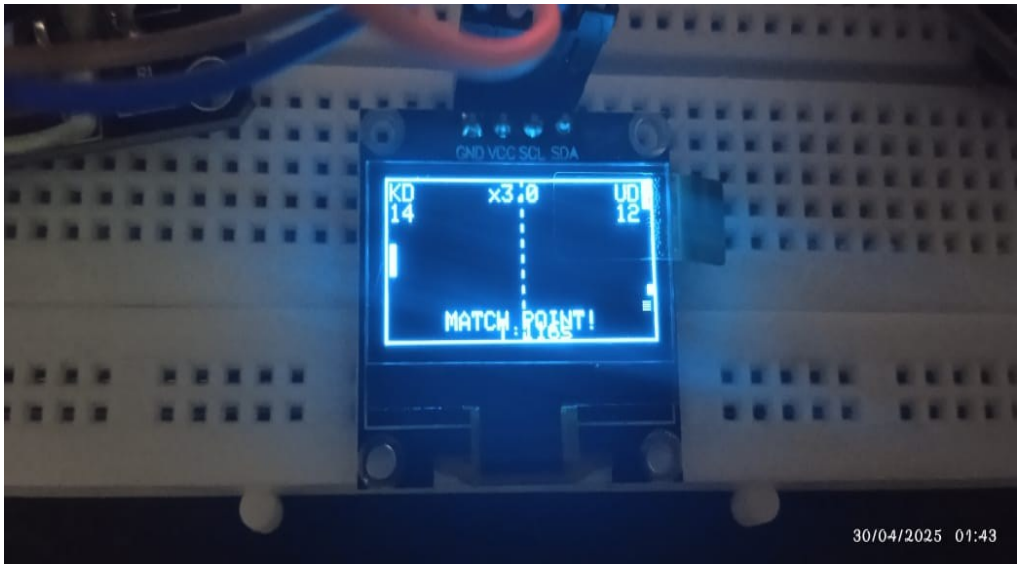

Figure 7: Running Model of the Project showcasing the matchpoint

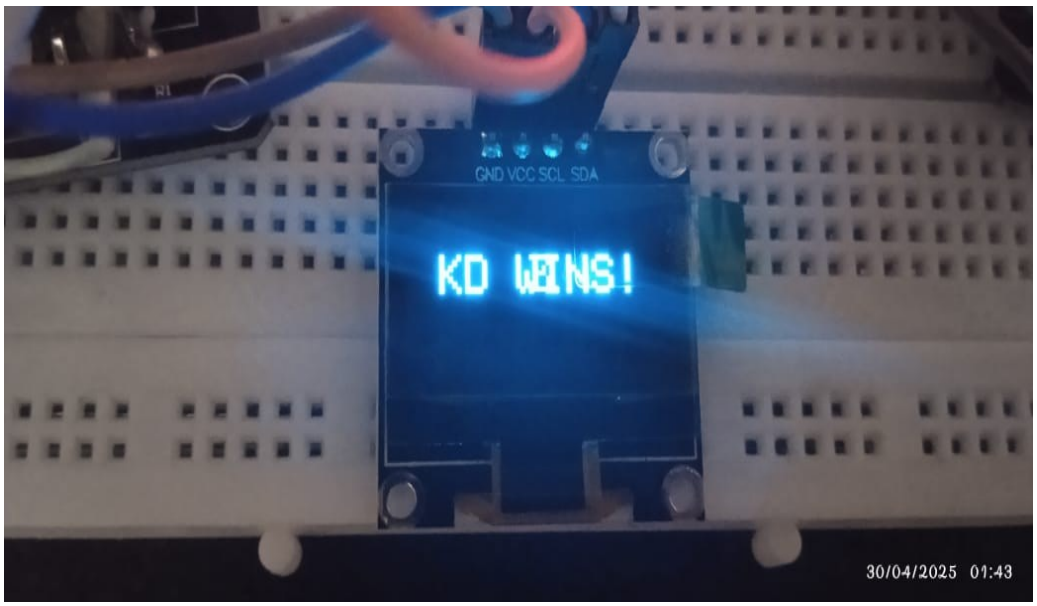

Figure 8: Output Screen after a player wins

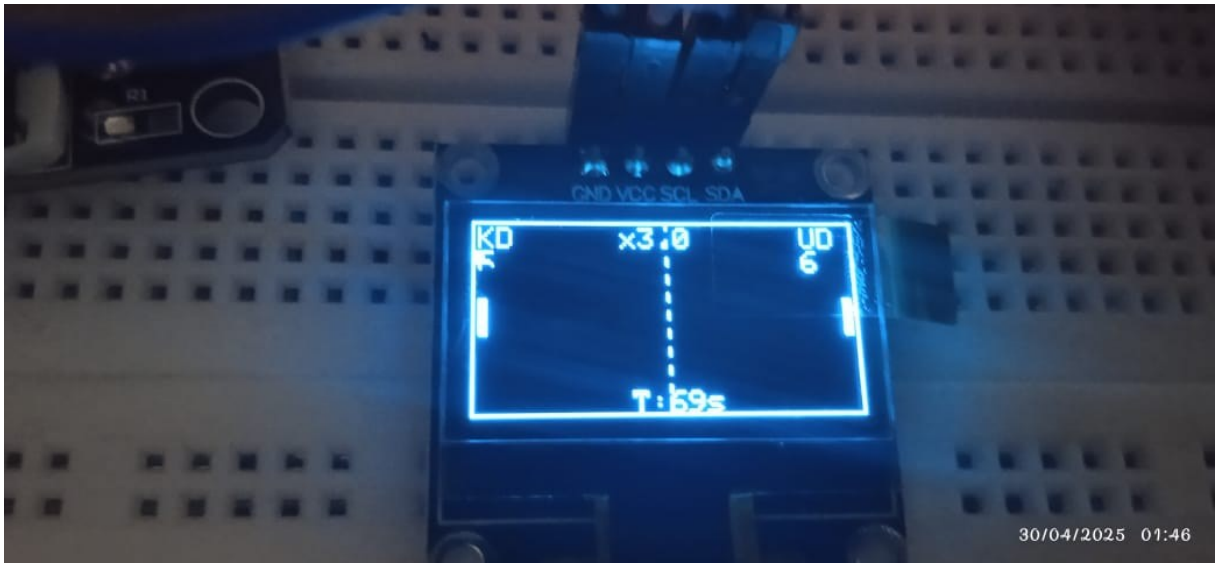

Figure 8: Front view of the OLED Display.

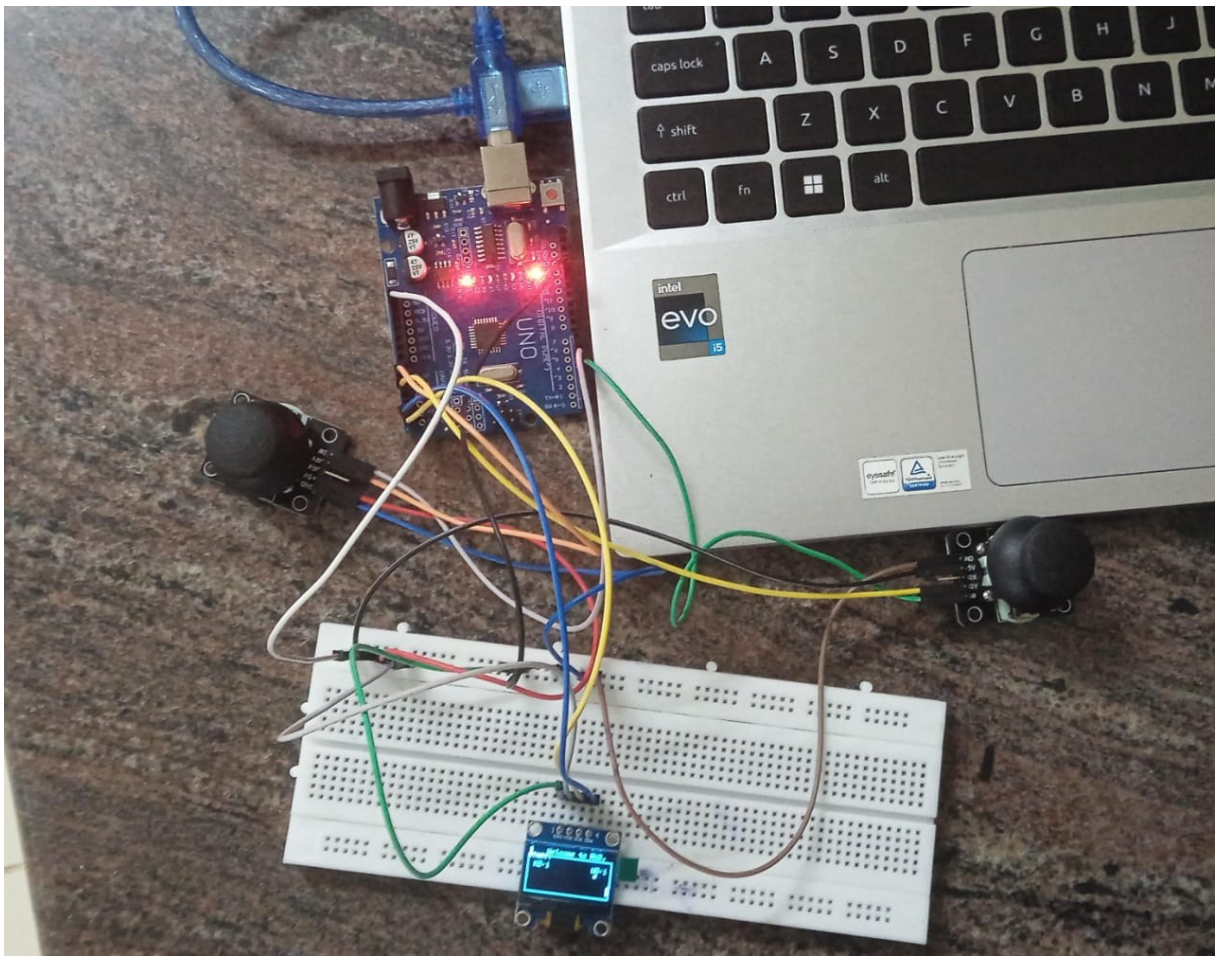

Figure 9: Top view of the Game Set-up

Applications:

- **Educational Projects:** An ideal learning platform to teach embedded game design, microcontroller interfacing, and graphics rendering.
- **Mini Gaming Console:** Serves as a base prototype for small arcade-like handheld games.
- **Workshops & Exhibits:** A practical and visually appealing project to display at academic fairs, workshops, or tech exhibitions.
- **Embedded Systems Testing:** Can be used as a benchmark to test timing, latency, and display performance in similar embedded systems.

Inference of the Mini Project:

From the implementation of the Pong Game using Arduino UNO and an OLED display, it is inferred that low-cost microcontrollers can effectively handle basic graphical operations and real-time game logic. The project revealed that even within limited memory and processing constraints, responsive gameplay, smooth animation, and hardware-level input/output handling are achievable. The interactive nature of the game proved useful in understanding microcontroller peripherals, digital communication, and user input detection. Furthermore, the optional buzzer feedback emphasized the role of multisensory interfaces in improving user experience.

Conclusion:

The Pong Game project serves as an innovative blend of classic entertainment and embedded systems learning. Utilizing Arduino UNO's simple features and the simplicity of the SSD1306 OLED, this project highlights the potential of microcontroller-based systems to run basic interactive applications. The minimal hardware requirement, low power consumption, and portable design make it ideal for educational and demonstration purposes. This project not only reinforces the basics of programming and interfacing in embedded development but also opens avenues for more complex graphical applications.

Future Advancements:

- Multiplayer via ESP-NOW, Wi-Fi, or Bluetooth
- Upgrade to color TFT display for richer visuals
- AI-controlled paddle for single-player mode
- Touch-based inputs for smoother control
- Game menu interface and persistent score saving

References:

General Sources and Technical References: (1)

1. Arduino UNO - https://www.arduino.cc/
2. Adafruit Industries. (2023). *Adafruit SSD1306 OLED Library Documentation*. https://learn.adafruit.com/monochrome-oled-breakouts/arduino-library-and-examples
3. Arduino Project Hub. (2022). *DIY Pong Game using OLED*. https://create.arduino.cc/projecthub
4. Rao, P., & Vyas, P. (2022). *A Review of ESP32 Applications in Embedded Systems Education*. *International Journal of Embedded Systems*, 13(4), 297–305. DOI: 10.1504/IJES.2022.10039921

Literature Survey Papers (2)

1. Restrepo-Alvarez, A. F., & Navas, D. F. (2014). Basic programming structure using a microcontroller system: Ping-Pong game. Link: revistaingenieria.univalle.edu.co
2. Saha, T., Ahmed, A. K. T., & Alam, S. M. S. (2023). Design and Implementation of Multi Player Pong Game using Altera DE2 Board. *International Journal of Intelligent Systems and Applications*, 15(6), 25-37. Link : MECS Press (https://www.mecs-press.org/ijisa/ijisa-v15-n6/v15n6-3.html)
3. Duch, P., & Jaworski, T. (2018). Enriching Computer Science Programming Classes with Arduino Game Development. *2018 11th International Conference on Human System Interaction (HSI)*. Link: ResearchGate (https://www.researchgate.net/publication/327006434_Enriching_Computer_Science_Programming_Classes_with_Arduino_Game_Development)
4. Logofătu, D. (2018). Machine Learning with the Pong Game: A Case Study. *19th International Conference on Engineering Applications of Neural Networks (EANN 2018)*. Link: ResearchGate (https://www.researchgate.net/publication/328403928_Machine_Learning_With_The_Pong_Game_A_Case_Study)
5. Ansari, A. T., Thivakaran, K., Kumar, M. H., & Renganathan, R. (2015). Design of a Smart Ping Pong Robot. *International Journal for Innovative Research in Science & Technology*, 1(10). Link: https://www.academia.edu/25694465/Design_of_a_Smart_Ping_Pong_Robot_Design_of_a_Smart_Ping_Pong_Robot